

AURORA Alpha Harness

面向递归自我改进的可审计 Alpha 研究系统技术报告

AutoAlpha Research Team

2026 年 9 月 8 日

AURORA = Auditable Unified Recursive Optimization and Research Architecture

AURORA 中的 RSI 指 Recursive Self-Improvement，而不是 Relative Strength Index。

Abstract

AURORA Alpha Harness 是一个将大语言模型、量化因子研究、统计验证和实验审计组合起来的研究基础设施。系统的核心对象不是单个收益率最高的表达式，而是一个可复现的研究过程：AI 提出机制假设，受限 DSL 将假设编译为可执行因子，冻结数据切分、成本和断言，运行测量与证伪检验，再根据结构化证据生成下一代研究任务。每一次迭代都绑定父结构、预算、数据哈希、代码哈希和结果状态，因此研究方法可以递归改进，同时不会获得修改评估器、读取未来数据或把未知结果伪装成通过的权限。

本文描述当前仓库中的完整架构，包括数据平面、结构目录、因子 DSL、研究流水线、honest R-learning 浅森林、自动演化队列、DeepSeek 角色防火墙、审计与恢复、组合 gate、企业 SDK 及 3D 控制台。报告也区分正式配置与 fast 配置：fast 用于快速验证 RSI 工程闭环，不具有正式确认资格。

Contents

1 系统定位与命名

1.1 为什么叫 AURORA Alpha Harness

AURORA Alpha Harness 的中文可称为“极光 Alpha 研究护栏”。AURORA 表示四个设计目标：

1. **Auditable:** 可审计。每个结构、模型请求、测量结果和失败原因都形成持久化记录。
2. **Unified:** 统一协议。同一个冻结配置贯穿数据读取、表达式编译、统计检验、回测和前端展示。
3. **Recursive:** 递归演化。研究结果只通过受控的 parent、operator 和 depth 字段影响下一代搜索。
4. **Optimization and Research Architecture:** 优化与研究架构。系统优化的是研究效率和研究假设，不允许 AI 自行改变评估规则。

这里的 RSI 明确表示 **Recursive Self-Improvement**。金融技术指标可以作为候选 alpha 的输入，但不是本项目的核心含义。AURORA 的创新点在于：AI 不直接宣称“这个因子有效”，而是在固定 harness 中提出下一轮可证伪的研究任务，并接受同一套评估器的检验。

1.2 面向评委与企业用户的叙事

企业研究员通常需要三件事：能接入自己的数据、能快速得到候选因子、能解释候选因子为什么被保留或丢弃。AURORA 将这三件事拆成三个层次：

Research engine 比赛引擎负责冻结协议、研究结构、探索和统计证据。

Research SDK 企业用户可以上传规范化 CSV/Parquet，定义研究格子，运行独立的 fast 或 balanced 实验。

Control panel 前端展示研究队列、父子血缘、森林发现、组合 gate、耗时和失败原因，适合现场演示与运维。

2 总体架构

2.1 分层设计

系统由六个相互隔离但通过机器可读产物连接的层组成：

1. 数据层：从只读 Parquet 构造按交易日和证券对齐的 codeMarketPanel，生成历史滚动字段和未来收益标签。
2. 假设层：由机制家族、结构形式、受控标签、可证伪断言和操作表达式组成 Structure。
3. 执行层：通过 DSL 编译、测量、placebo、稳定性、成本和条件发现产生研究证据。
4. 演化层：依据证据生成 horizontal、cross-family、forest、crossover 和 probe 任务。
5. 治理层：通过 DeepSeek 角色防火墙、代码/数据哈希、SQLite 审计链、检查点和恢复脚本控制边界。
6. 产品层：通过 SDK、FastAPI、控制面板和组合回放向研究员展示和复用结果。

2.2 代码模块映射

模块	当前职责
engine/data.py, research_fields.py engine/config.py	数据扫描、字段派生、标签构造、行业和竞价质量边界。
engine/dsl.py, forms.py, catalog.py engine/pipeline.py engine/placebo.py, blades.py, metrics.py engine/discovery.py	日期边界、成本、随机种子、检验规模、模式和 worker 数的冻结配置。 受限表达式编译、结构形式契约、机制家族和可搜索网络。 单结构生命周期、检查点、测量提交、后处理和批次完成。 安慰剂、断言检验、HAC/块 bootstrap、稳定性和成本分析。 A 段候选筛查、交叉拟合正交化、honest R-learning 浅森林、ICM 交互矩。
engine/discovery_parallel.py, engine/cycle.py, continuation.py engine/llm.py	流模式按时间切分的 Windows spawn 多进程发现。 任务队列、自动演化、父结构继承和续接批次。 DeepSeek proposer、reviewer、bettor、reconciler、inducer 的角色隔离与缓存。
engine/audit.py composition/ research_sdk/ studio_api.py src/, dashboard_server.py	SQLite 事件链、结构冻结、文件哈希和访问凭证。 多结构组合 gate、成分快照和组合回放。 企业数据上传、格子定义、独立运行和报告导出。 React/Three.js 控制面板、研究进度和 3D 研究场景。

3 数据平面与时间边界

3.1 只读面板

原始 Parquet 永不被研究过程修改。数据层首先依据日期和证券谓词扫描输入，再构造长表和矩阵双视图：矩阵视图适合滚动字段、分组收益和前端图表，长表视图适合 DSL 结果、森林训练和逐观测审计。

MarketPanel 包含两组对象：

- **fields**: 只依赖当日及历史的价格、成交、规模、行业、指数快照、波动和估值代理；
- **labels**: 从 T+1 开盘到 T+1+h 开盘生成的未来收益和行业残差标签。

字段缺失、无效比率、暖启动不足、行业缺失和异常竞价状态保持为 NaN 或隔离状态，不被静默填成“其他”。字段和缓存身份包含实现源码哈希，避免同一文件大小掩盖代码变化。

3.2 A/B/H 研究分段

当前探索数据范围为 2016-07-01 至 2022-06-30。系统还预留确认段 B（2022-07-22 至 2025-07-31）和 holdout 段 H（2025-08-01 至 2026-07-29），但 fast 模式不读取 B/H 作为正式确认依据。

结构内部采用时间顺序：A 段先进行测量和条件搜索；所有搜索候选冻结后，才允许相应的验证检验。训练与验证之间使用 **purge**，避免未来收益窗口和滚动特征跨边界泄漏。

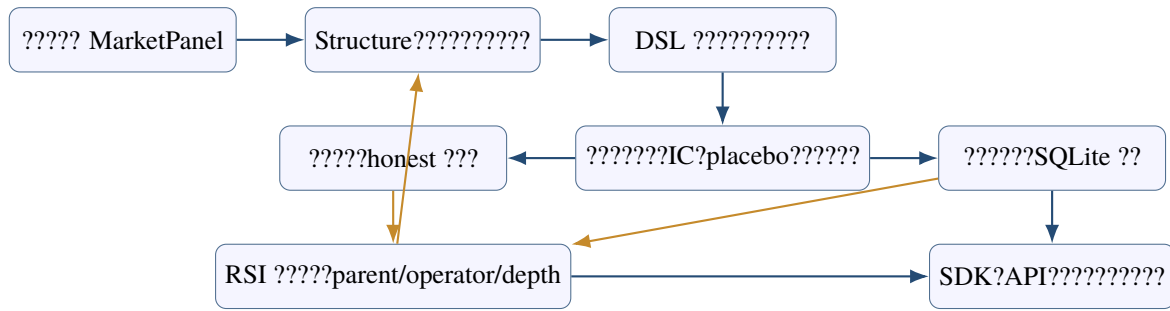


Figure 1: AURORA 的研究与产品分层。演化层只能生成新任务，不能改写冻结评估层。

4 研究结构：从格子到可证伪档案

4.1 Structure 是最小研究单位

一个 Structure 不是一条表达式，而是一份完整档案：

$$\text{Structure} = (\text{coordinate}, \text{labels}, \text{assertions}, \text{operational}, \text{lineage}, \text{freeze hash}). \quad (1)$$

coordinate 机制家族 M1–M10 与结构形式 F1–F7 的坐标。

labels 从固定词表选择 agent、friction、mispricing、correction、form 五个槽位。

assertions 测量前冻结的 P1、P2、P3 及条件断言；例如符号、峰值窗口、剂量形状和 side condition。

operational 同一机制的三个可执行表达式，用于检查收敛效度，降低单表达式过拟合。

lineage parent、operator、moderator、donor、depth 和来源记录。

断言状态只有三种：**hold**（支持）、**violated**（与预测矛盾）、**untested**（证据不足）。**untested** 不被当成失败，也不允许 LLM 根据它编造机制规律。

4.2 受限 DSL

表达式通过白名单函数、字段注册表、窗口边界和方向检查编译。编译器阻止：未来字段、未知列、非法窗口、错误量纲、没有可观测事件的事件机制以及未登记的 operator。LLM 只能生成候选规格，不能执行任意 Python。

典型表达式例如：

```
neg(ts_z(ret_1d, 20))
xs_rank(ret_20d)
neg(ts_std(ret_1d, 20))
```

这些表达式只是研究候选，不代表已经通过正式确认。

5 单结构运行管线

一个结构从提案到归档经过以下阶段：

1. 从 seed 或演化队列取得坐标和父结构。
2. DeepSeek proposer 根据受控词表和 form contract 生成机制、表达式、方向和峰值范围。
3. reviewer 检查逻辑可行性；它看不到研究结果。
4. DSL 编译、字段登记和 convergent orientation 检查。

5. 生成结构 ID，计算表达式、覆盖和配置指纹。
6. bettor 在测量前对断言盲下注，概率独立落盘。
7. 测量 IC、分组曲线、月度曲线、贡献分解、HAC、覆盖和成本。
8. 运行 placebo、assertion、stability、increment 等检验。
9. 对探索候选运行条件发现和 ICM 交互检验。
10. 提交 result、research-factor、daily-ic、contribution 和耗时。
11. reconciler 只负责把确定性断言结果分类为 aligned/divergent/unresolved；无效响应会被拒绝并可审计恢复。
12. 根据当前证据生成有限 follow-up，进入下一轮队列。

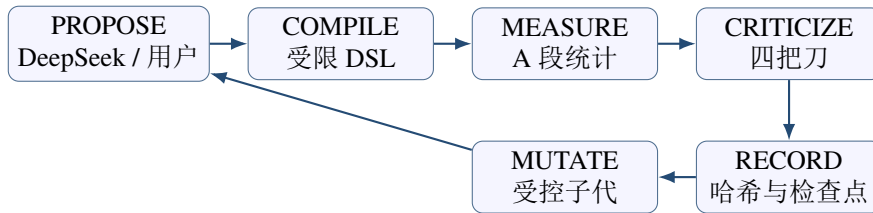


Figure 2: AURORA 的 RSI 六步闭环。

6 条件发现与多进程森林

6.1 为什么需要条件发现

因子可能只在某个规模、波动、成交或行业状态下成立。AURORA 不允许对全样本任意切分后挑最好的一组，而是先在 A 段进行候选筛查，再使用固定切点、purged split 和独立验证路径检验条件结构。

6.2 honest R-learning 浅森林

对每个候选 moderator，系统构造：

$$Y_i = r_i - \hat{m}(X_i), \quad D_i = f_i - \hat{g}(X_i), \quad (2)$$

其中 r_i 是未来行业残差收益, f_i 是研究信号, X_i 是规模、波动、成交和行业控制变量。orthogonalize() 在 purged folds 内拟合稀疏行业指标、三次多项式、样条和 Ridge nuisance model，然后计算残差。

森林只在冻结切点上生成浅树。每棵树使用独立日期块和样本权重，叶子必须满足独立日期和证券覆盖要求。森林输出变量选择频率和叶子诊断，不向 LLM 直接提供“因果效应方向”。

6.3 Windows 多进程实现

fast 模式的两个时间切分相互独立，因此使用 Windows spawn 子进程并行计算：

- 子进程通过临时 Parquet 读取对应日期范围，不序列化整个 MarketPanel；
- 每个进程内的 BLAS、树训练和数值库线程限制为 1，避免嵌套并发；
- 按可用内存动态决定实际 worker 数，内存不足时自动退化为串行；
- 结果按切分索引合并，保持随机种子、p 值和候选频率与串行结果一致。

在同一台机器的全量冻结结构上，候选筛查加发现阶段从 454.923 秒降至 238.610 秒，约 1.907 倍加速；两折 p 值和候选频率逐值一致。该数字是阶段级同机复核，不是对所有机器的固定承诺。

7 RSI 自动演化闭环

7.1 预算与队列

--full-grid 当前建立 70 个初始 seed 任务，并预留 140 次自动演化预算，总尝试上限为 210。140 不是预先写死的 140 个表达式，而是后续队列可以消耗的预算；每完成一个结构，系统最多追加一个直接 follow-up，实际数量取决于可验证证据和可用格子。

队列任务具有如下字段：

operator	含义	主要触发条件
seed	网格冷启动	尚未研究的机制/形式坐标
horizontal	同家族换形式	shape/sign 断言被违反或覆盖不足
cross_family	竞争机制	side 断言被违反
forest	条件门控	森林频率高且探索 p 值满足 fast 阈值
crossover	条件迁移	已有结构支持的 side moderator
probe	记忆引导探针	inducer 提出有 law 证据的未探索坐标

每个子代继承父任务的问题上下文，而不是父结构的结论。子代必须重新进行表达式编译、断言冻结、盲下注和完整测量，最大 lineage depth 为 2。父结构是失败、未知还是支持，会分别影响可生成的动作；fast 模式允许探索性子代继续验证，但不授予正式 law 或 PASS。

7.2 RSI 的严格边界

AURORA 的“自我改进”有四个约束：

1. AI 不能修改 evaluator、时间切分、成本、数据准入和正式确认门槛；
2. AI 不能读取尚未冻结的验证或 holdout 结果来修改当前结构；
3. AI 只能提交结构化 JSON，并由 schema、DSL 和审计层拒绝非法结果；
4. 演化是有限预算的研究搜索，不是无限自我复制或无边界模型升级。

8 DeepSeek 角色与信息防火墙

DeepSeek 不是一个拥有完整数据库权限的“自动交易员”，而是五个受限角色：

proposer 根据坐标、词表、form contract 和父结构先验提出机制规格。

reviewer 只审核先验逻辑、字段可用性和形式一致性，不读取结果。

bettor 在测量前对断言给出盲先验概率。

reconciler 将已经计算好的断言状态分组，不能改变状态。

inducer 根据已经落盘的 law 提出 probe 和连接假设，不能宣布确认。

每次请求以 role、context、instruction、model、nonce 和版本形成缓存身份。缓存既提高恢复速度，也让同一请求可复核。2026 年 9 月 8 日第二结构曾收到三个空数组的 reconciler 响应，系统拒绝该响应；恢复脚本保留原响应、使用唯一 nonce 重试，并在确认完整覆盖后恢复运行。这体现了“模型输出可失败，但研究证据不能被默默篡改”的原则。

9 审计、检查点与可复现性

9.1 冻结身份

每个批次的 identity 包含：

- 配置、日期、成本、随机种子、模式和 worker 上限；
- Python 与关键依赖版本；
- 输入 Parquet 的文件哈希；
- engine 源码哈希；
- DeepSeek endpoint/model 的公开配置身份；
- discovery 与 Bayesian 开关。

如果继续运行时 identity 变化，系统拒绝恢复；如果要改变代码、统计规模或数据范围，必须创建新批次。continue-from 只继承已验证结构和队列，不把继承结果冒充新批次的独立重复。

9.2 检查点提交顺序

结构的测量和结果先写入结构目录，再把 pending_postprocess 写入 checkpoint。若 LLM 后处理、记忆刷新或 inducer 失败，恢复时只重做后处理，不重复昂贵的测量和森林。每个恢复事件也进入 SQLite 事件链。

10 正式、工程与 fast 模式

模式	主要用途	当前协议
formal	正式确认	bootstrap 1000、placebo 500、森林 500 棵树、20 次切分；需要 B/H 独立确认。
engineering fast	工程验收 RSI 闭环与演示	缩小重复次数，验证端到端流程和数据边界。 全量 A 段、19 次诊断 placebo、30 棵树、2 次切分、多进程；允许探索性子代，但禁止正式 PASS。

fast 的目标是尽快回答“提案、编译、测量、批评、演化、组合和恢复能否闭环”，而不是用缩小的重复次数取得正式统计结论。fast 报告中的 p 值只作为资源分配和研究诊断，不能写成最终因子显著性。

11 组合 Gate 与多结构研究

单结构完成后，composition/ 可以按固定完成顺序冻结前 4、8、16 个结构，生成组合 gate 和回放。组合过程不接收益临时挑成分，不把组合结果回写成单结构的验证证据。组合输出包括成分快照、暴露、冲突持仓、成本和执行状态。

这种设计允许研究叙事从“AI 找到一个因子”升级为“AI 形成了可审计的结构组合”：

1. 基础结构提供机制和表达式；
2. 条件发现提供 gate；
3. crossover 迁移可复用条件；
4. composition runner 评估多结构相互作用；
5. 回放结果仍与正式确认资格分离。

12 企业 SDK 与控制面板

12.1 SDK

`research_sdk` 面向企业用户提供轻量化入口：

1. 上传 CSV/Parquet 并校验 `date`、`symbol`、`open`、`close` 和数值字段；
2. 以 `CellSpec` 定义机制、表达式和 `horizon`；
3. 以 `ExperimentSpec` 定义 `fast/balanced`、训练比例、成本、演化开关和预算；
4. 固定数据哈希、代码哈希和验证边界；
5. 运行训练段演化、验证 IC、`block bootstrap`、成本敏感性和相似度检查；
6. 输出 `candidates`、`factors`、`daily path`、`quality status` 和 `artifact hashes`。

SDK 的输出状态是 `NEEDS_REVIEW` 或 `REPLICATION_CANDIDATE`，不会把单次用户运行直接包装成正式金融结论。

12.2 控制面板

React/Three.js 控制台包含研究队列、运行模式、数据上传、结构卡、进度日志、3D 研究场景、父子血缘、候选表和组合视图。3D 场景的作用是把机制家族、结构形式、代际关系和运行状态可视化，而不是把统计结果装饰成确定结论。前端通过 `FastAPI` 读取结构化产物，不绕过后端检查直接修改研究状态。

13 当前实现状态与已知边界

截至 2026 年 9 月 8 日，仓库已经实现：

- 70 格初始网格、140 次演化预算和最大深度 2 的自动演化队列；
- `DeepSeek` 五角色研究编排与响应缓存；
- A 段测量、`placebo`、断言、稳定性、增量、成本和条件发现；
- `Windows` 多进程森林，阶段级复核约 1.907 倍加速且统计输出一致；
- `SQLite` 审计、哈希冻结、检查点恢复和 `reconciler` 无效响应恢复；
- 组合 `gate`、回放、企业 SDK、`FastAPI` 和 3D 控制面板。

当前仍需明确的边界包括：

- `fast` 模式不具备正式确认资格；
- 研究区间中的数据隔离、行业来源和竞价数据质量仍会影响正式资格；
- 自动演化预算是上限，不保证产生同等数量的有效子代；
- LLM 是受限研究代理，不是统计检验器；
- 当前组合回放不能替代独立 B/H 确认和真实交易接入。

14 一次完整运行的操作顺序

```
# 1. fast RSI / autoresearch harness
.venv-research/Scripts/python.exe -u run_engine.py \
--fast --full-grid --workers 2 \
--industry-policy quarantine \
--industry-source exact_intervals \
--auction-policy quarantine \
--run-id aurora-fast-YYYYMMDD

# 2. 查看批次状态和研究摘要
.venv-research/Scripts/python.exe scripts/summarize_research.py \
artifacts/aurora-fast-YYYYMMDD

# 3. 启动控制面板
.venv-research/Scripts/python.exe dashboard_server.py
```

运行完成后，评审者应首先查看：checkpoint.json、queue.json、law.md、overview.json、结构目录中的 frozen.json/result.json，以及组合目录中的 gate 和回放报告。这样可以先检查研究过程，再查看候选收益。

15 结论

AURORA Alpha Harness 的核心贡献是把“AI 自我改进”从口号变成了受约束的研究协议。AI 可以提出机制、选择下一代任务、组合条件并改进研究路线；评估器、时间切分、成本、数据权限、预算和正式确认门槛仍由 harness 控制。每一次改进都有父结构、操作符、深度、输入哈希、运行状态和失败记录，因此评委可以追问“为什么生成这个结构”“它看到了哪些信息”“哪一步被拒绝”“是否重用了验证结果”，系统能够给出可复核的答案。

这使 AURORA 同时具备三种形态：比赛中的全量 alpha research engine、企业内部的因子研究 SDK，以及面向 AI agent 的 RSI autoresearch harness。它的目标不是让模型无限地产生漂亮表达式，而是让研究方法在有限预算和严格审计下，稳定地产生更清晰、更可证伪、更容易复现的下一轮研究问题。

A 术语表

术语	含义
RSI	Recursive Self-Improvement，递归自我改进；本项目不是 Relative Strength Index。
Harness	固定评估协议、权限和审计边界的研究护栏。
Structure	坐标、标签、断言、表达式、血缘和冻结哈希构成的研究档案。
A/B/H	探索、确认和 holdout 时间分段。
Placebo	检查效应是否可能由伪影、前视或巧合造成的诊断。
UNDECIDABLE	证据不足以判定，不等于 FAIL。
Forest	在冻结切点和 purged 时间切分上进行条件发现的 honest 浅森林。
Gate	由 moderator 或条件变量控制结构暴露的门函数。
Lineage depth	子代距离根结构的演化深度，当前最大为 2。